شرح سطر بسطر ١٨ — محرّك الجولة وتسجيل التقدّم

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

`RoundEngine` هو قلب التدريب والاختبار: دوال نقية بلا واجهة ولا قاعدة بيانات. الشاشة تحتفظ بـ `RoundState` غير قابل للتغيير وتستدعي `submit` و`advance` فتحصل على حالة جديدة. لهذا اختُبر بالكامل بلا أي ودجت (21 اختبارًا).

القواعد التي اعتمدناها: **التدريب** — ردّ فوري، وكل إجابة خاطئة تُظهر تلميحًا جديدًا (ويُسمح بالمحاولة ثانية)، وبعد نفاد التلميحات يُكشف الحل ويُحتسب السؤال خطأ. **الاختبار** — محاولة واحدة لكل سؤال بلا تلميحات ولا ردّ حتى النهاية.

القيم: صح من أول مرة = 3 نجوم وإتقان 1؛ صح بعد تلميح = نجمتان وإتقان 0.5؛ خطأ = صفر. الإتقان لكل مفهوم متوسط متحرّك (0.6 القديم + 0.4 الجديد). تقييم النجوم الثلاث يتبع نسبة الإتقان (30% و60% و90%).

`LearningRepository` يكتب النتيجة كلها في معاملة واحدة: المحاولة بتفاصيل كل سؤال (JSON) وتحديث الإتقان وإنجاز جزء الدرس، فلا يبقى نصف مكتوب لو انقطع التطبيق. وتُمنع تكرارات النجوم: ما نِيل من نشاط لا يُمنح ثانية.

lib/features/learning/round_engine.dart — محرّك الجولة

lib/features/learning/round_engine.dart

الأسطر 1–3

```dart
import 'dart:math';

import '../content/question.dart';
```

ماذا يفعل؟ استيراد `dart:math` للخلط و`min`، ونماذج الأسئلة.

لماذا بهذه الطريقة؟ لا استيراد لـ Flutter إطلاقًا: الملف منطق صرف.

الأسطر 5–6

```dart
/// منطق جولة الأسئلة (تدريب أو اختبار) كدوال نقية بلا واجهة ولا قاعدة بيانات،
/// وتستدعي الدوال هنا فقط [RoundState] ليسهل اختباره. الشاشة تحتفظ بـ
```

ماذا يفعل؟ تعليق يشرح أن الملف دوال نقية، والشاشة تحمل الحالة.

لماذا بهذه الطريقة؟ يوضّح التقسيم: المنطق هنا، والعرض في الشاشات.

الأسطر 8–14

```dart
enum RoundMode {
  /// تدريب: ردّ فوري، تلميحات متدرّجة بعد كل خطأ، ثم الحل.
  practice,

  /// اختبار: محاولة واحدة لكل سؤال، بلا تلميحات ولا ردّ حتى النهاية.
  quiz,
}
```

ماذا يفعل؟ وضعا الجولة: تدريب واختبار وقواعد كل منهما.

لماذا بهذه الطريقة؟ وضع واحد للمحرّك يغيّر السلوك بدل نسختين منه.

الأسطر 16–25

```dart
enum RoundPhase {
  /// ينتظر إجابة (وقد تكون تلميحات ظاهرة بعد خطأ سابق).
  answering,

  /// أجاب صحيحًا (تدريب)، ننتظر «السؤال التالي».
  correct,

  /// انتهت محاولاته بلا نجاح (تدريب): يُعرض الحل، ثم «السؤال التالي».
  revealed,
}
```

ماذا يفعل؟ مراحل السؤال الحالي: ينتظر إجابة، أجاب صح، أو كُشف الحل.

لماذا بهذه الطريقة؟ المراحل الثلاث هي ما تحتاجه الشاشة لتقرر ماذا تعرض (لوحة نجاح أو حل أو تلميح) وهل تقبل لمسات.

الأسطر 27–38

```dart
/// ترتيب العرض لسؤال بعد الخلط. الإجابة تبقى بأرقام العناصر الأصلية دائمًا،
/// فلا يحتاج منطق التصحيح معرفة ما خُلط.
class QuestionLayout {
  const QuestionLayout({this.order = const [], this.cards = const []});

  /// موضع العرض ← رقم العنصر الأصلي: الخيارات، أو الجانب الأيمن في التوصيل،
  /// أو بداية ترتيب العناصر.
  final List<int> order;

  /// ترتيب بطاقات السحب والإفلات.
  final List<String> cards;
}
```

ماذا يفعل؟ `QuestionLayout`: ترتيب العرض بعد الخلط: `order` لأرقام العناصر الأصلية و `cards` لبطاقات السحب.

لماذا بهذه الطريقة؟ الجواب يبقى بالأرقام الأصلية دائمًا، فالتصحيح لا يعرف شيئًا عن الخلط ولا يمكن أن يخطئ بسببه.

الأسطر 40–57

```dart
class QuestionResult {
  const QuestionResult({
    required this.questionId,
    required this.concept,
    required this.kind,
    required this.correct,
    required this.wrongs,
    required this.hints,
  });

  factory QuestionResult.fromJson(Map<String, dynamic> json) => QuestionResult(
    questionId: json['id'] as String,
    concept: json['concept'] as String,
    kind: QuestionKind.values.byName(json['kind'] as String),
    correct: json['correct'] as bool,
    wrongs: json['wrongs'] as int,
    hints: json['hints'] as int,
  );
```

ماذا يفعل؟ نتيجة سؤال: حقولها ومُنشئ `fromJson` .

لماذا بهذه الطريقة؟ تُحفظ في JSON داخل المحاولة، وتُقرأ لاحقًا لشاشتي النتيجة والمراجعة.

الأسطر 59–70

```dart
  final String questionId;
  final String concept;
  final QuestionKind kind;
  final bool correct;

  /// عدد الإجابات الخاطئة قبل النهاية.
  final int wrongs;

  /// عدد التلميحات التي رآها الطالب (والحل يُحسب تلميحًا).
  final int hints;

  bool get firstTry => correct && wrongs == 0;
```

ماذا يفعل؟ الحقول: المعرّف والمفهوم والنوع وصح/خطأ وعدد الأخطاء والتلميحات، ثم `firstTry` .

لماذا بهذه الطريقة؟ تلميحات الحل تُحتسب تلميحًا واحدًا لتلخيص «كم مساعدة احتاج».

الأسطر 72–94

```
  /// قيمة الإتقان لهذا السؤال: 1 من أول مرة، 0.5 بعد تلميح، 0 خطأ.
  double get value => !correct ? 0 : (wrongs == 0 ? 1 : 0.5);

  /// نجوم السؤال (نظام الألعاب في التصميم): 3 من أول محاولة، 2 بعد التلميح
  /// الأول، 1 بعد الثاني أو أكثر، 0 إن لم يُحَل. لا تُخصم نجوم أبدًا.
  int get stars => !correct ? 0 : starsFor(wrongs);

  /// نجوم الحل الصحيح بعد [wrongs] محاولات خاطئة.
  static int starsFor(int wrongs) => switch (wrongs) {
    0 => 3,
    1 => 2,
    _ => 1,
  };

  Map<String, dynamic> toJson() => {
    'id': questionId,
    'concept': concept,
    'kind': kind.name,
    'correct': correct,
    'wrongs': wrongs,
    'hints': hints,
  };
}
```

ماذا يفعل؟ قيمة الإتقان (1 / 0.5 / 0)، ونجوم السؤال عبر 3 : `starsFor(wrongs)` من أول محاولة، 2 بعد التلميح الأول، 1 بعد الثاني أو أكثر، و0 إن لم يُحَل. ثم `toJson` .

لماذا بهذه الطريقة؟ قاعدة «نظام الألعاب» في التصميم الجديد (كانت 3 ثم 2 بعد أي عدد من التلميحات). `starsFor` دالة ثابتة لأن شاشة الجولة تحتاجها أيضًا لعنوان «أحسنت!» قبل أن يكتمل `QuestionResult` . لا تُخصم نجوم أبدًا.

الأسطر 96–106

```
class RoundState {
  const RoundState({
    required this.mode,
    required this.questions,
    required this.layouts,
    this.results = const [],
    this.index = 0,
    this.wrongs = 0,
    this.phase = RoundPhase.answering,
    this.lastAnswer,
  });
```

ماذا يفعل؟ `RoundState` : الوضع والأسئلة وتخطيطاتها ونتائج المنتهي والفهرس والأخطاء والمرحلة وآخر إجابة.

لماذا بهذه الطريقة؟ حالة كاملة غير قابلة للتغيير؛ كل خطوة تنتج نسخة جديدة فيسهل الاختبار ولا تتلاعب الواجهة بها.

الأسطر 108–121

```
  final RoundMode mode;
  final List<Question> questions;
  final List<QuestionLayout> layouts;

  /// نتائج الأسئلة المنتهية بالترتيب.
  final List<QuestionResult> results;
  final int index;

  /// الإجابات الخاطئة على السؤال الحالي.
  final int wrongs;
  final RoundPhase phase;

  /// آخر إجابة قُدّمت للسؤال الحالي (للتلوين بعد التصحيح).
  final Object? lastAnswer;
```

ماذا يفعل؟ حقول الحالة بتوثيقها.

لماذا بهذه الطريقة؟ `lastAnswer` يلوّن البطاقة الخاطئة أو الصحيحة بعد التصحيح.

الأسطر 123–125

```
  bool get finished => index >= questions.length;
  Question? get current => finished ? null : questions[index];
  QuestionLayout? get layout => finished ? null : layouts[index];
```

ماذا يفعل؟ `finished` و`current` و`layout` للسؤال الحالي (null عند الانتهاء).

لماذا بهذه الطريقة؟ تمنع الوصول بفهرس خارج النطاق.

الأسطر 127–132

```
  /// عدد التلميحات الظاهرة الآن (تلميح لكل خطأ، بحدّ عدد التلميحات).
  int get hintsShown {
    final q = current;
    if (q == null || mode == RoundMode.quiz) return 0;
    return min(wrongs, q.hints.length);
  }
```

ماذا يفعل؟ `hintsShown`: عدد التلميحات الظاهرة = عدد الأخطاء بحدّ عدد التلميحات، وصفر في الاختبار.

لماذا بهذه الطريقة؟ تلميح واحد لكل خطأ: يتدرّج الدعم ولا يُعطى الحل دفعة.

الأسطر 134–143

```
  /// هل الحل الكامل ظاهر؟
  bool get solutionShown =>
      phase == RoundPhase.revealed && (current?.solution.isNotEmpty ?? false);

  /// هل يُسمح بالضغط على «الحل خطوة بخطوة» مبكرًا؟
  bool get canRevealSolution =>
      mode == RoundMode.practice &&
      phase == RoundPhase.answering &&
      (current?.solution.isNotEmpty ?? false) &&
      wrongs > 0;
```

ماذا يفعل؟ `solutionShown` و`canRevealSolution`: متى الحل ظاهر، ومتى يُسمح بطلبه مبكرًا (تدريب، بعد خطأ، وللسؤال حل).

لماذا بهذه الطريقة؟ لا زر «التلميح الأخير» قبل أول خطأ؛ فالطالب يجرّب أولًا.

الأسطر 145–164

```
  RoundState copyWith({
    List<QuestionResult>? results,
    int? index,
    int? wrongs,
    RoundPhase? phase,
    Object? lastAnswer,
    bool clearAnswer = false,
  }) {
    return RoundState(
      mode: mode,
      questions: questions,
      layouts: layouts,
      results: results ?? this.results,
      index: index ?? this.index,
      wrongs: wrongs ?? this.wrongs,
      phase: phase ?? this.phase,
      lastAnswer: clearAnswer ? null : (lastAnswer ?? this.lastAnswer),
    );
  }
}
```

ماذا يفعل؟ `copyWith` لإنتاج حالة جديدة، و`clearAnswer` لتصفير آخر إجابة (لأن `null` يعني «لا تغيير»).

لماذا بهذه الطريقة؟ النمط المعتاد للحالات الثابتة مع حيلة صغيرة لفرق «لا تغيير» و«صفّر».

الأسطر 166–180

```
abstract final class RoundEngine {
  /// أقصى أسئلة جولة التدريب.
  static const practiceLength = 8;

  /// أقصى أسئلة جولة التدريب للصفوف 1–4 (نظام الألعاب في التصميم).
  static const kidsPracticeLength = 5;

  /// آخر صف يأخذ جولة الصغار (5 أسئلة وبلا «رتّب»).
  static const lastKidsGrade = 4;

  /// أقصى أسئلة الاختبار.
  static const quizLength = 10;

  /// عتبات نجوم التقييم (3..0) بحسب نسبة الإتقان.
  static const _starThresholds = [0.3, 0.6, 0.9];
```

ماذا يفعل؟ بداية `RoundEngine`: أطوال الجولات (8 تدريب و10 اختبار)، وجولة الصغار (`kidsPracticeLength = 5` حتى `lastKidsGrade = 4`)، وعتبات التقييم.

لماذا بهذه الطريقة؟ ثوابت مسمّاة بدل أرقام سحرية. أطفال الصفوف 1–4 يتعبون من جولة طويلة، فقاعدة التصميم: 5 أسئلة.

الأسطر 182–206

```
  /// يحصر [concept] و[practiceLength] الأسئلة المختارة. التدريب: خلط ثم أول
  /// الأسئلة في مفهوم واحد إن وُجد له أسئلة). الاختبار: ترتيب المؤلّف كما هو.
  ///
  /// [kids] (الصفوف 1–4): التدريب [kidsPracticeLength] رتّب «أسئلة بلا»،
  /// إلا إن لم يبق غيرها.
  static List<Question> pick(
    List<Question> pool,
    RoundMode mode, {
    required Random random,
    String? concept,
    bool kids = false,
  }) {
    if (mode == RoundMode.quiz) return pool.take(quizLength).toList();
    var source = pool;
    if (concept != null) {
      final only = pool.where((q) => q.concept == concept).toList();
      if (only.isNotEmpty) source = only;
    }
    if (kids) {
      final noOrder = source.where((q) => q is! OrderQuestion).toList();
      if (noOrder.isNotEmpty) source = noOrder;
    }
    final copy = [...source]..shuffle(random);
    return copy.take(kids ? kidsPracticeLength : practiceLength).toList();
  }
```

ماذا يفعل؟ `pick`: الاختبار بترتيب المؤلّف حتى 10. التدريب يحصر في مفهوم إن طُلب، ثم للصغار (`kids`) يستبعد «رتّب» إن بقي غيرها، ثم يخلط ويأخذ 5 للصغار أو 8.

لماذا بهذه الطريقة؟ الاختبار متدرّج بترتيب المعلم، والتدريب متنوع. «رتّب» يحتاج سحبًا دقيقًا صعبًا على الصغار (قاعدة التصميم: بلا «رتّب» للصفوف 1–4)، لكن إن لم يوجد غيرها نبقيها حتى لا تصير الجولة فارغة.

الأسطر 208–218

```
  static RoundState start({
    required RoundMode mode,
    required List<Question> questions,
    required Random random,
  }) {
    return RoundState(
      mode: mode,
      questions: questions,
      layouts: [for (final q in questions) _layoutFor(q, random)],
    );
  }
```

ماذا يفعل؟ `start`: يبني الحالة الأولى ويحسب تخطيطًا لكل سؤال.

لماذا بهذه الطريقة؟ الخلط يُحسب مرة عند البداية؛ فلا يتغيّر ترتيب الخيارات عند إعادة بناء الشاشة.

الأسطر 220–227

```
  /// لا تُخلط «كل ما سبق»/«لا شيء مما سبق»… — خيارات تعتمد على ترتيبها.
  static bool keepsOrder(List<String> choices) => choices.any(
    (c) =>
        c.contains('كل ما سبق') ||
        c.contains('جميع') ||
        c.contains('كلاهما') ||
        c.contains('لا شيء مما'),
  );
```

ماذا يفعل؟ `keepsOrder`: خيارات مثل «كل ما سبق» لا تُخلط.

لماذا بهذه الطريقة؟ هذه الخيارات تفقد معناها إن لم تكن أخيرة.

الأسطر 229–240

```
  static QuestionLayout _layoutFor(Question q, Random random) {
    List<int> shuffled(int n, {bool avoidIdentity = false}) {
      final base = List.generate(n, (i) => i);
      if (n < 2) return base;
      final list = [...base];
      var tries = 0;
      do {
        list.shuffle(random);
        tries++;
      } while (avoidIdentity && _isIdentity(list) && tries < 10);
      return list;
    }
```

ماذا يفعل؟ `_layoutFor` — الدالة المحلية `shuffled`: تخلط 0..n-1، وتُعيد المحاولة حتى 10 مرات إن طُلب تجنّب الترتيب الأصلي.

لماذا بهذه الطريقة؟ سؤال ترتيب يبدأ مرتّبًا صحيحًا أصلًا سيكون بلا معنى.

الأسطر 242–262

```
    return switch (q) {
      ChoiceQuestion(:final choices) => QuestionLayout(
        order: keepsOrder(choices)
            ? List.generate(choices.length, (i) => i)
            : shuffled(choices.length),
      ),
      ListenChooseQuestion(:final choices) => QuestionLayout(
        order: shuffled(choices.length),
      ),
      MatchPairsQuestion(:final pairs) => QuestionLayout(
        order: shuffled(pairs.length, avoidIdentity: true),
      ),
      OrderQuestion(:final items) => QuestionLayout(
        order: shuffled(items.length, avoidIdentity: true),
      ),
      DragDropQuestion(:final cards) => QuestionLayout(
        cards: [...cards]..shuffle(random),
      ),
      TrueFalseQuestion() => const QuestionLayout(),
    };
  }
```

ماذا يفعل؟ `switch` على نوع السؤال: يُخلط الاختيار (إلا لو احتفظ بالترتيب) والاستماع والتوصيل والترتيب والبطاقات، أما صح/خطأ فلا شيء.

لماذا بهذه الطريقة؟ لأن `Question` مغلق (`sealed`) فالمترجم يضمن أن كل نوع مغطّى.

الأسطر 264–269

```
  static bool _isIdentity(List<int> l) {
    for (var i = 0; i < l.length; i++) {
      if (l[i] != i) return false;
    }
    return true;
  }
```

ماذا يفعل؟ `_isIdentity`: هل القائمة هي الترتيب الأصلي؟

لماذا بهذه الطريقة؟ مساعدة لتجنّب بداية صحيحة.

الأسطر 271–291

```
271   /// يقدّم إجابة السؤال الحالي.
272   ///
273   /// - اختبار: تُسجَّل النتيجة فورًا وننتقل للتالي (لا ردّ ولا إعادة).
274   /// - تدريب: صحيحة ← [RoundPhase.correct]. خاطئة ← تلميح جديد ويُسمح بالمحاولة
275   ///   ثانية؛ وبعد استنفاد التلميحات ← [RoundPhase.revealed] (الحل ظاهر).
276   static RoundState submit(RoundState s, Object? answer) {
277     final q = s.current;
278     if (q == null || s.phase != RoundPhase.answering) return s;
279     final ok = q.check(answer);
280
281     if (s.mode == RoundMode.quiz) {
282       return s.copyWith(
283         results: [
284           ...s.results,
285           _result(q, correct: ok, wrongs: ok ? 0 : 1, hints: 0),
286         ],
287         index: s.index + 1,
288         wrongs: 0,
289         clearAnswer: true,
290       );
291     }
```

ماذا يفعل؟ `submit` — الفحوص ثم وضع الاختبار: تُسجّل النتيجة فورًا وننتقل للسؤال التالي.

لماذا بهذه الطريقة؟ الاختبار بلا ردّ ولا إعادة، ولا يُحتسب فيه تلميح.

الأسطر 292–304

```
292     if (ok) {
293       return s.copyWith(phase: RoundPhase.correct, lastAnswer: answer);
294     }
295     final wrongs = s.wrongs + 1;
296     return s.copyWith(
297       wrongs: wrongs,
298       lastAnswer: answer,
299       phase: wrongs > q.hints.length
300           ? RoundPhase.revealed
301           : RoundPhase.answering,
302     );
303   }
```

ماذا يفعل؟ `submit` — التدريب: الصحيح ينقل إلى `correct`؛ الخطأ يزيد العدّاد ويكشف الحل إذا فاق عدد الأخطاء عدد التلميحات.

لماذا بهذه الطريقة؟ بدون تلميحات: أول خطأ يكشف الجواب. بتلميحين: الخطأ الثالث. هكذا لا تُترك الطالب بلا نهاية.

الأسطر 306–310

```
306   /// الطالب يطلب الحل مبكرًا (زر «التلميح الأخير»).
307   static RoundState revealSolution(RoundState s) {
308     if (!s.canRevealSolution) return s;
309     return s.copyWith(phase: RoundPhase.revealed);
310   }
```

ماذا يفعل؟ `revealSolution`: كشف الحل مبكرًا إن سُمح.

لماذا بهذه الطريقة؟ يحترم `canRevealSolution`.

الأسطر 312–328

```
312   /// ينهي السؤال الحالي (بعد صح أو كشف الحل) وينتقل للتالي.
313   static RoundState advance(RoundState s) {
314     final q = s.current;
315     if (q == null || s.phase == RoundPhase.answering) return s;
316     final correct = s.phase == RoundPhase.correct;
317     final hints = min(s.wrongs, q.hints.length) + (s.solutionShown ? 1 : 0);
318     return s.copyWith(
319       results: [
320         ...s.results,
321         _result(q, correct: correct, wrongs: s.wrongs, hints: hints),
322       ],
323       index: s.index + 1,
324       wrongs: 0,
325       phase: RoundPhase.answering,
326       clearAnswer: true,
327     );
328   }
```

ماذا يفعل؟ `advance`: بعد الصح أو الكشف تُسجَّل نتيجة السؤال (تلميحات = الظاهر + الحل إن عُرض) وننتقل.

لماذا بهذه الطريقة؟ نسجّل عند الانتقال لا عند الإجابة، لأن حالة الكشف المبكر والصح تنتهيان هنا.

الأسطر 330–342

```
330   static QuestionResult _result(
331     Question q, {
332     required bool correct,
333     required int wrongs,
334     required int hints,
335   }) => QuestionResult(
336     questionId: q.id,
337     concept: q.concept,
338     kind: q.kind,
339     correct: correct,
340     wrongs: wrongs,
341     hints: hints,
342   );
```

ماذا يفعل؟ `_result`: يبني `QuestionResult`.

لماذا بهذه الطريقة؟ تقليل تكرار بناء النتيجة في مسارين.

الأسطر 344–346

```
344   /// الإتقان بعد سؤال جديد: متوسط متحرّك. أول مرة تُؤخذ قيمة السؤال كما هي.
345   static double nextMastery(double? old, double value) =>
346       old == null ? value : (old * 0.6 + value * 0.4).clamp(0.0, 1.0);
```

ماذا يفعل؟ `nextMastery`: أول قياس يؤخذ كما هو ثم متوسط متحرّك 0.6/0.4 محصور بين 0 و1.

لماذا بهذه الطريقة؟ يُثبّت المفهوم تدريجيًا: خطأ واحد لا يمحو إتقانًا سابقًا، وتحسّن مستمر يرفعه.

الأسطر 348–358

```
  /// تقييم النجوم من 0 إلى 3 بحسب نسبة الإتقان (مجموع قيم الأسئلة ÷ العدد).
  static int ratingStars(List<QuestionResult> results) {
    if (results.isEmpty) return 0;
    final ratio =
        results.fold<double>(0, (a, r) => a + r.value) / results.length;
    var stars = 0;
    for (final t in _starThresholds) {
      if (ratio >= t) stars++;
    }
    return stars;
  }
```

ماذا يفعل؟ `ratingStars`: عدد العتبات التي بلغتها نسبة الإتقان (حتى 3).

لماذا بهذه الطريقة؟ نجوم الشاشة تقييم الأداء، وتختلف عن «النجوم» العملة.

الأسطر 360–365

```
  /// النسبة المئوية الصحيحة (أسئلة أُجيبت صح ÷ الكل).
  static int percent(List<QuestionResult> results) {
    if (results.isEmpty) return 0;
    return (results.where((r) => r.correct).length * 100 / results.length)
        .round();
  }
```

ماذا يفعل؟ `percent`: نسبة الإجابات الصحيحة.

لماذا بهذه الطريقة؟ تُعرض في شاشة النتيجة وتحدد نجاح الاختبار.

الأسطر 367–370

```
  /// مجموع «نجوم» الجولة (عملة المكافأة).
  static int starsEarned(List<QuestionResult> results) =>
      results.fold(0, (a, r) => a + r.stars);
}
```

ماذا يفعل؟ `starsEarned`: مجموع نجوم الجولة.

لماذا بهذه الطريقة؟ العملة التي تُصرف في الخزانة.

lib/features/learning/learning_repository.dart — كتابة التقدّم وتسجيل الجولات

`lib/features/learning/learning_repository.dart`

الأسطر 1–5

```
import 'dart:convert';

import '../../core/db/app_database.dart';
import '../content/lesson_content.dart';
import 'round_engine.dart';
```

ماذا يفعل؟ استيراد JSON وقاعدة البيانات ومحتوى الدرس والمحرّك.

لماذا بهذه الطريقة؟ المستودع يربط المنطق النقي بالقاعدة.

الأسطر 7–15

```
/// ما سُجّل بعد انتهاء جولة. [attemptId] يُفتح به شاشة النتيجة.
class RoundRecord {
  const RoundRecord({required this.attemptId, required this.earned});

  final int attemptId;

  /// النجوم الجديدة فعلًا (بعد خصم ما نالته المحاولات السابقة لنفس النشاط).
  final int earned;
}
```

ماذا يفعل؟ `RoundRecord`: ما يعود بعد التسجيل: معرّف المحاولة والنجوم الجديدة فعلًا.

لماذا بهذه الطريقة؟ المعرّف يُفتح به شاشة النتيجة، و`earned` يعرض «+N».

الأسطر 17–26

```
/// تفاصيل محاولة محفوظة (detailsJson) بعد فكّها.
class AttemptDetails {
  const AttemptDetails({
    required this.lessonId,
    required this.mode,
    required this.results,
    required this.rawStars,
    required this.masteryBefore,
    required this.masteryAfter,
  });
```

ماذا يفعل؟ `AttemptDetails`: تفاصيل محاولة بعد فكّها وحقولها.

لماذا بهذه الطريقة؟ تعيد بناء الجولة لشاشتي نهاية الجولة والنتيجة من القاعدة نفسها.

الأسطر 28–45

```
  factory AttemptDetails.fromJson(String raw) {
    final j = jsonDecode(raw) as Map<String, dynamic>;
    Map<String, double?> mastery(String key) => {
      for (final e in (j[key] as Map<String, dynamic>).entries)
        e.key: (e.value as num?)?.toDouble(),
    };
    return AttemptDetails(
      lessonId: j['lessonId'] as String,
      mode: RoundMode.values.byName(j['mode'] as String),
      results: [
        for (final r in j['questions'] as List<dynamic>)
          QuestionResult.fromJson(r as Map<String, dynamic>),
      ],
      rawStars: j['rawStars'] as int,
      masteryBefore: mastery('masteryBefore'),
      masteryAfter: mastery('masteryAfter'),
    );
  }
```

ماذا يفعل؟ قراءة JSON: دالة محلية `mastery` تحوّل خريطة الإتقان (قيمة أو null)، ثم بناء الكائن.

لماذا بهذه الطريقة؟ تحويل `num?` إلى `double?` لأن JSON يحفظ 1 صحيحًا لا عشريًا.

الأسطر 47–57

```
  final String lessonId;
  final RoundMode mode;
  final List<QuestionResult> results;

  /// نجوم الجولة قبل خصم المكرّر.
  final int rawStars;

  /// إتقان كل مفهوم قبل الجولة وبعدها (null = لم يُقَس).
  final Map<String, double?> masteryBefore;
  final Map<String, double?> masteryAfter;
}
```

ماذا يفعل؟ حقول التفاصيل، منها إتقان كل مفهوم قبل الجولة وبعدها.

لماذا بهذه الطريقة؟ يسمح بعرض «40% ← 65%».

الأسطر 59–69

```
/// كتابة تقدّم الدروس: إنهاء الأجزاء وحساب حالة الدرس ونسبته منها، وتسجيل جولات
/// التدريب والاختبار.
class LearningRepository {
  LearningRepository(this._db, {DateTime Function()? now})
    : _now = now ?? DateTime.now;

  final AppDatabase _db;
  final DateTime Function() _now;

  /// أدنى نسبة نجاح ليُعدّ الاختبار منجزًا.
  static const quizPassPercent = 60;
```

ماذا يفعل؟ المستودع وحقوله و`quizPassPercent` (60).

لماذا بهذه الطريقة؟ حدّ نجاح الاختبار ثابت واحد.

الأسطر 71–72

```
  static String activityId(String lessonId, RoundMode mode) =>
      '$lessonId.${mode.name}';
```

ماذا يفعل؟ `activityId`: معرّف نشاط الدرس بصيغة `<الدرس>.<الوضع>`.

لماذا بهذه الطريقة؟ يجمع محاولات التدريب في نشاط وكذلك الاختبار، ويُستعمل في منع تكرار النجوم.

الأسطر 74–89

```
  /// يعلّم [part] منجزًا، ثم يعيد حساب تقدّم الدرس: النسبة = الأجزاء المنجزة ÷
  /// هو محتوى الدرس (للمعرفة [content] المتوفرة للدرس، و«مكتمل» عند إنجاز كلها
  /// الأجزاء المتوفرة)؛ بدونه يبقى الدرس «قيد التعلّم» بنسبة تقديرية.
  Future<void> completePart({
    required String profileId,
    required String lessonId,
    required LessonPart part,
    required LessonContent? content,
  }) async {
    final now = _now();
    await _db.activityDao.markPartDone(
      profileId: profileId,
      lessonId: lessonId,
      part: part,
      now: now,
    );
```

ماذا يفعل؟ `completePart` — يعلّم الجزء منجزًا في القاعدة.

لماذا بهذه الطريقة؟ التكرار لا يضرّ (مرة واحدة).

الأسطر 90–105

```
    final done = {
      for (final p in await _db.activityDao.getLessonParts(profileId))
        if (p.lessonId == lessonId) p.part,
    };
    final available = content?.parts ?? [part];
    final finished = available.where(done.contains).length;
    final total = available.isEmpty ? 1 : available.length;
    final fraction = (finished / total).clamp(0.0, 1.0);
    await _db.activityDao.upsertLessonProgress(
      profileId: profileId,
      lessonId: lessonId,
      status: fraction >= 1 ? LessonStatus.completed : LessonStatus.inProgress,
      progress: fraction,
      now: now,
    );
  }
```

ماذا يفعل؟ حساب النسبة: المنجز ÷ المتوفر للدرس، ثم كتابة تقدّم الدرس: «مكتمل» عند 1 وإلا «قيد التعلّم».

لماذا بهذه الطريقة؟ الحساب من الأجزاء المتوفرة فعلًا، فلا يعلق درس لأن فيه جزءًا غير موجود.

الأسطر 107–120

```
  /// يسجّل جولة منتهية: المحاولة (مع تفاصيل كل سؤال)، تحديث إتقان المفاهيم،
  /// وإنجاز جزء التدريب (أو الاختبار إن بلغ [quizPassPercent]).
  /// كل ذلك في معاملة واحدة فلا يبقى نصف مكتوب عند الانقطاع.
  Future<RoundRecord> recordRound({
    required String profileId,
    required String lessonId,
    required RoundMode mode,
    required List<QuestionResult> results,
    required LessonContent? content,
  }) {
    return _db.transaction(() async {
      final now = _now();
      final dao = _db.activityDao;
      final activity = activityId(lessonId, mode);
```

ماذا يفعل؟ بداية `recordRound` داخل معاملة قاعدة بيانات.

لماذا بهذه الطريقة؟ المعاملة تضمن أن المحاولة والإتقان والإنجاز تُكتب كلها أو لا شيء.

الأسطر 122–125

```
      // نجوم المحاولة لا تتجاوز ما لم يُنَل بعد من هذا النشاط (لا تكرار للمكافأة).
      final raw = RoundEngine.starsEarned(results);
      final already = await dao.starsForActivity(profileId, activity);
      final earned = (raw - already).clamp(0, raw);
```

ماذا يفعل؟ النجوم: ناتج الجولة ناقص ما نِيل من نفس النشاط سابقًا (ولا ينزل عن صفر).

لماذا بهذه الطريقة؟ يمنع تكرار الجولة لجمع النجوم نفسها؛ يُكافأ التحسّن فقط.

الأسطر 127–149

```
      // الإتقان: متوسط متحرّك بترتيب الأسئلة.
      final current = {
        for (final m in await dao.getMastery(profileId)) m.conceptId: m.mastery,
      };
      final before = <String, double?>{};
      final after = <String, double?>{};
      final running = {...current};
      for (final r in results) {
        before.putIfAbsent(r.concept, () => current[r.concept]);
        running[r.concept] = RoundEngine.nextMastery(
          running[r.concept],
          r.value,
        );
        after[r.concept] = running[r.concept];
      }
      for (final e in after.entries) {
        await dao.upsertMastery(
          profileId: profileId,
          conceptId: e.key,
          mastery: e.value!,
          now: now,
        );
      }
```

ماذا يفعل؟ الإتقان: قراءة القائم، ثم المرور على الأسئلة بالترتيب لتطبيق المتوسط المتحرّك وتسجيل «قبل» و«بعد»، ثم كتابة كل مفهوم.

لماذا بهذه الطريقة؟ نحتفظ بـ«قبل» لأول ظهور للمفهوم، و«بعد» لآخر قيمة.

الأسطر 151–169

```
      final correct = results.where((r) => r.correct).length;
      final id = await dao.insertAttempt(
        profileId: profileId,
        activityId: activity,
        kind: mode == RoundMode.quiz ? AttemptKind.quiz : AttemptKind.practice,
        score: correct,
        total: results.length,
        starsEarned: earned,
        hintsUsed: results.fold(0, (a, r) => a + r.hints),
        detailsJson: jsonEncode({
          'lessonId': lessonId,
          'mode': mode.name,
          'rawStars': raw,
          'questions': [for (final r in results) r.toJson()],
          'masteryBefore': before,
          'masteryAfter': after,
        }),
        now: now,
      );
```

ماذا يفعل؟ إدخال المحاولة: النوع والنتيجة والنجوم والتلميحات وتفاصيل كل سؤال بصيغة JSON.

لماذا بهذه الطريقة؟ التفاصيل تبني شاشات النتيجة والمراجعة دون جداول إضافية.

الأسطر 171–184

```
      final passed =
          mode == RoundMode.practice ||
          RoundEngine.percent(results) >= quizPassPercent;
      if (passed) {
        await completePart(
          profileId: profileId,
          lessonId: lessonId,
          part: mode == RoundMode.quiz ? LessonPart.quiz : LessonPart.practice,
          content: content,
        );
      }
      return RoundRecord(attemptId: id, earned: earned);
    });
  }
```

ماذا يفعل؟ إنجاز الجزء: التدريب دائمًا، والاختبار بنسبة 60% فأكثر. ثم إعادة `RoundRecord` .

لماذا بهذه الطريقة؟ اختبار بنتيجة ضعيفة لا يُعدّ إنجازًا؛ فتبقى حلقة المراجعة قائمة.

الأسطر 186–210

```
  /// يسجّل جولة لعبة (بلا درس): محاولة فقط، بنجوم محدودة بسقف إجمالي للّعبة.
  Future<RoundRecord> recordGame({
    required String profileId,
    required String activityId,
    required List<QuestionResult> results,
    required int starsCap,
  }) {
    return _db.transaction(() async {
      final dao = _db.activityDao;
      final raw = RoundEngine.starsEarned(results);
      final already = await dao.starsForActivity(profileId, activityId);
      final earned = raw.clamp(0, (starsCap - already).clamp(0, starsCap));
      final id = await dao.insertAttempt(
        profileId: profileId,
        activityId: activityId,
        kind: AttemptKind.practice,
        score: results.where((r) => r.correct).length,
        total: results.length,
        starsEarned: earned,
        hintsUsed: results.fold(0, (a, r) => a + r.hints),
        now: _now(),
      );
      return RoundRecord(attemptId: id, earned: earned);
    });
  }
```

ماذا يفعل؟ `recordGame` : لعبة بلا درس؛ محاولة فقط والنجوم محدودة بسقف إجمالي للّعبة (`starsCap`).

لماذا بهذه الطريقة؟ لعبة العدّ بلا نهاية سيكون أسهل مصدر نجوم؛ السقف يحفظ قيمة الخزانة.

الأسطر 212–221

```
  /// محاولة محفوظة بتفاصيلها، أو null إن لم توجد أو كانت قديمة بلا تفاصيل.
  Future<({Attempt attempt, AttemptDetails details})?> loadAttempt(
    String profileId,
    int attemptId,
  ) async {
    final a = await _db.activityDao.getAttempt(profileId, attemptId);
    final raw = a?.detailsJson;
    if (a == null || raw == null) return null;
    return (attempt: a, details: AttemptDetails.fromJson(raw));
  }
```

ماذا يفعل؟ `loadAttempt`: يقرأ محاولة طالب بعينه وتفاصيلها، أو null.

لماذا بهذه الطريقة؟ التقييد بمعرّف الطالب يمنع قراءة محاولة طالب آخر.

السطر 222

```
}
```

ماذا يفعل؟ إغلاق الصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/learning/lesson_view.dart — درس بسياقه

lib/features/learning/lesson_view.dart

الأسطر 1–9

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../core/db/enums.dart';
import '../catalog/catalog_models.dart';
import '../catalog/catalog_providers.dart';
import '../content/content_providers.dart';
import '../content/lesson_content.dart';
import '../profiles/active_profile_provider.dart';
import 'learning_providers.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟ يجمع الكتالوج والمحتوى والملف النشط والمستودع.

الأسطر 11–26

```dart
/// درس واحد بكل ما تحتاجه شاشات أجزائه: الدرس ووحدته ومادته ومحتواه.
class LessonView {
  const LessonView({
    required this.lesson,
    required this.unit,
    required this.subject,
    required this.content,
  });

  final CatalogLesson lesson;
  final CatalogUnit unit;
  final CatalogSubject subject;

  /// null = لا ملف محتوى لهذا الدرس بعد.
  final LessonContent? content;
}
```

ماذا يفعل؟ `LessonView`: الدرس ووحدته ومادته ومحتواه (قد يكون null).

لماذا بهذه الطريقة؟ كل شاشة جزء تحتاج هذه الأربعة؛ نحمّلها مرة بدل أربع استدعاءات.

الأسطر 28–45

```dart
final lessonViewProvider = FutureProvider.family<LessonView?, String>((
  ref,
  lessonId,
) async {
  final catalog = await ref.watch(catalogProvider.future);
  final lesson = catalog.lessonById(lessonId);
  if (lesson == null) return null;
  final unit = catalog.unitOfLesson(lesson);
  final subject = catalog.subjectById(lesson.subjectId);
  if (unit == null || subject == null) return null;
  final content = await ref.watch(lessonContentProvider(lessonId).future);
  return LessonView(
    lesson: lesson,
    unit: unit,
    subject: subject,
    content: content,
  );
});
```

ماذا يفعل؟ `lessonViewProvider`: يقرأ الكتالوج ويبحث عن الدرس والوحدة والمادة، ثم المحتوى، ويرجع null إن غاب أي عنصر.

لماذا بهذه الطريقة؟ `FutureProvider.family` يخزّن النتيجة لكل درس ويشاركها بين الشاشات.

الأسطر 47–65

```
/// يعلّم جزءًا من درس منجزًا للملف النشط ويحدّث تقدّم الدرس. لا يفعل شيئًا
/// بلا ملف نشط.
Future<void> completeLessonPart(
  WidgetRef ref, {
  required String lessonId,
  required LessonPart part,
}) async {
  final profileId = ref.read(activeProfileIdProvider).value;
  if (profileId == null) return;
  final content = await ref.read(lessonContentProvider(lessonId).future);
  await ref
      .read(learningRepositoryProvider)
      .completePart(
        profileId: profileId,
        lessonId: lessonId,
        part: part,
        content: content,
      );
}
```

ماذا يفعل؟ `completeLessonPart`: يعلّم جزءًا منجزًا للملف النشط (لا شيء بلا ملف).

لماذا بهذه الطريقة؟ دالة واحدة تستدعيها كل الشاشات عند الإنجاز.